

LIBRERÍA DE COMUNICACIONES

DESCRIPCIÓN

Dentro de esta librería hay 3 clases principales: El servidor, el cliente TCP y el cliente UDP. El servidor es capaz de aceptar “infinitas” conexiones TCP y UDP simultáneamente. Además cada petición recibida se tramita en un hilo diferente para poder hacer varias peticiones en paralelo.

Existen 5 tipos de peticiones distintas:

- Petición de tipo byte[] con respuesta. Bloqueante hasta respuesta o TimeOut.
- Petición de tipo byte[] sin respuesta. No bloqueante.
- Petición de tipo string con respuesta. Bloqueante hasta respuesta o TimeOut.
- Petición de tipo string sin respuesta. No bloqueante.
- Ping. Bloqueante hasta respuesta o TimeOut.

Además estos 5 tipos de peticiones se pueden hacer en los dos sentidos: Cliente a servidor y servidor a cliente. En el caso de que el cliente sea TCP no existe ningún tipo de problema, pero cuando el cliente es de tipo UDP el servidor solo almacena la información del cliente durante 300 segundos desde su última comunicación. Si no se quiere perder la conexión UDP se deben hacer un PING o una petición antes de que transcurra este tiempo.

Cuando el servidor hace peticiones a los clientes de tipo sin respuesta se pueden hacer 2 cosas: Si no se indica a que cliente se quiere enviar la información el paquete se enviará a todos los clientes TCP y UDP. Si se quiere hacer la petición a un cliente en concreto hay que decirle a cual.

Cuando el servidor hace peticiones a los clientes de tipo con respuesta se debe indicar forzosamente a que cliente se quiere hacer la petición.

El PING se debe hacer a un cliente en concreto. Cuando la función termina devuelve -1 si no ha habido respuesta o un valor positivo que es el número de milisegundos que ha tardado en llegar la respuesta.

Todas las clases tienen un evento llamado “Exception” que se dispara cuando ocurre una excepción dentro de la clase para poder tratarla.

Todos los paquetes que se envían están comprimidos, si ocupan menos que el paquete original, y encriptado con el protocolo AES (Rijndael):

http://es.wikipedia.org/wiki/Advanced_Encryption_Standard

El tamaño máximo del paquete a enviar depende mucho de la compresión conseguida y es difícil de determinar, pero si el paquete no se puede enviar por exceso de tamaño o por no estar conectada la función de hacer petición devolverá “falso”.

Nombre de espacio:

Creta.libcomhmi

SERVER

Recibe conexiones de tipo TCP y UDP por el mismo puerto. El puerto de se debe indicar en el constructor y se mantendrá abierto.

Propiedades:

- **Puerto:** Propiedad de solo lectura de tipo Int32. Indica el puerto que se está usando como entrada.

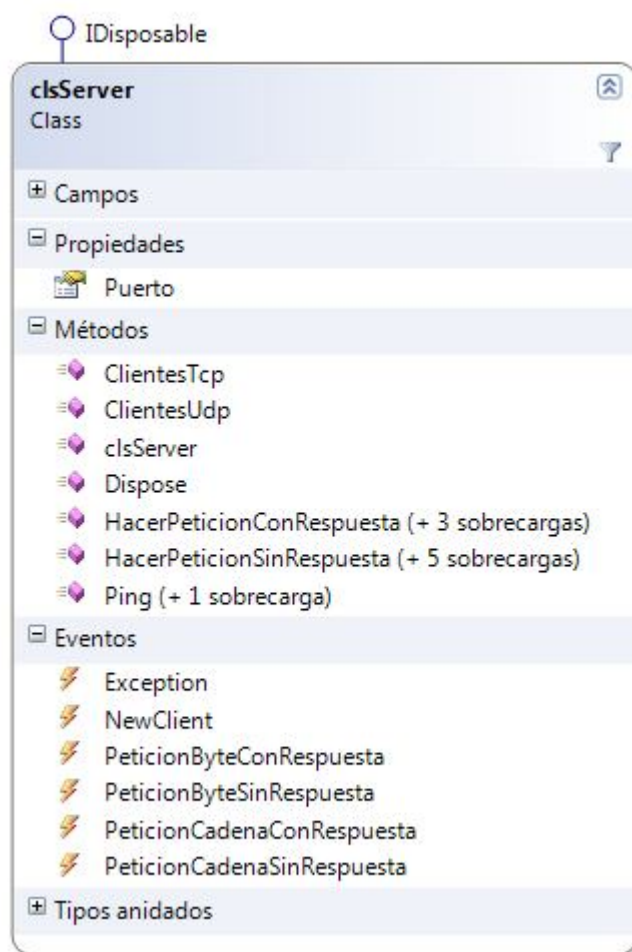
Métodos:

- **Clientes TCP:** Devuelve una lista con todos los clientes TCP conectados actualmente.
- **Clientes UDP:** Devuelve una lista con todos los clientes UDP conectados actualmente.
- **Dispose:** Finaliza el servidor. Cierra todas las conexiones abiertas y termina los hilos internos de forma controlada.
- **HacerPeticiónConRespuesta:** Hace una petición a un cliente en concreto TCP o UDP y espera la respuesta del cliente. Puede hacer peticiones de 2 tipos: De cadena de texto o de matriz de bytes. Este método se bloquea hasta que llegue la respuesta o hasta que se cumpla el Timeout indicado.
- **HacerPeticiónSinRespuesta:** Hace peticiones a todos los clientes a la vez si no se indica ningún cliente, o a un cliente en concreto de tipo TCP o UDP. Puede hacer dos tipos de peticiones: De cadena de texto o de matriz de bytes. Este método no se bloquea.
- **Ping:** Hace un Ping a un cliente TCP o UDP en concreto y devuelve el tiempo que ha tardado en llegar la respuesta en milisegundos. Este método se bloquea hasta que llega la respuesta o hasta que se cumple el Timeout indicado. Si no hay respuesta o la petición falla devuelve -1.

Eventos:

- **Exception:** Este evento avisa de un error interno.
- **NewClient:** Este evento avisa de la entrada de un nuevo cliente. Cuando un cliente TCP se conecta o un cliente UDP envía su primer petición se dispara una vez para aceptar o no al cliente. Una vez aceptado no se vuelve a disparar hasta que el cliente no se desconecte y se vuelva a conectar.

- **PeticionByteConRespuesta:** Este evento avisa de que ha llegado un paquete de tipo matriz de bytes desde un cliente y además hay que devolver una respuesta al cliente del mismo tipo.
- **PeticionByteSinRespuesta:** Este evento avisa de que ha llegado un paquete de tipo matriz de bytes desde un cliente pero no se devuelve ninguna respuesta.
- **PeticionCadenaConRespuesta:** Este evento avisa de que ha llegado un paquete de tipo cadena de caracteres desde un cliente y además hay que devolver una respuesta al cliente del mismo tipo.
- **PeticionCadenaSinRespuesta:** Este evento avisa de que ha llegado un paquete de tipo cadena de caracteres desde un cliente pero no se devuelve ninguna respuesta.



CLIENTE TCP

Conecta con el servidor utilizando el protocolo TCP. Este protocolo es el recomendado ya que no tiene ningún tipo de limitación con el servidor.

Propiedades:

- **Conectado:** Propiedad de solo lectura de tipo Bool. Indica que la conexión con el servidor es correcta.
- **Tiempo:** Propiedad de solo lectura de tipo Int32. Devuelve la media de tiempo de peticiones en milisegundos de las últimas 10 peticiones.

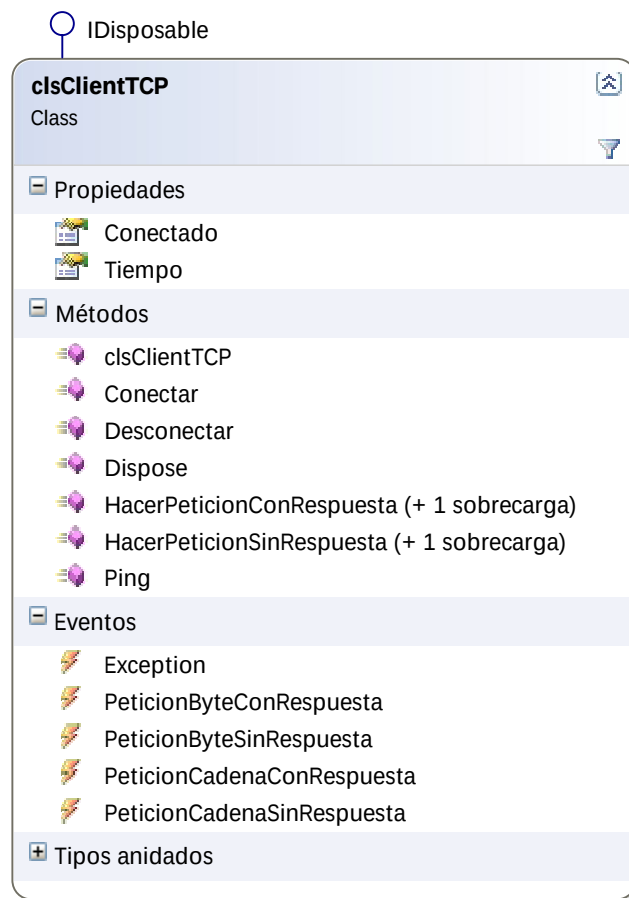
Métodos:

- **Conectar:** Se conecta a un servidor. Solo hay que indicar la dirección y puerto.
- **Desconectar:** Se desconecta del servidor.
- **Dispose:** Finaliza el cliente. Cierra la conexión abierta y termina los hilos internos de forma controlada.
- **HacerPeticiónConRespuesta:** Hace una petición al servidor y espera la respuesta. Puede hacer peticiones de 2 tipos: De cadena de texto o de matriz de bytes. Este método se bloquea hasta que llegue la respuesta o hasta que se cumpla el Timeout indicado. Se pueden hacer tantas peticiones en paralelo en distintos hilos como sea necesario.
- **HacerPeticiónSinRespuesta:** Hace una petición al servidor. Puede hacer dos tipos de peticiones: De cadena de texto o de matriz de bytes. Este método no se bloquea. Se pueden hacer tantas peticiones en paralelo en distintos hilos como sea necesario.
- **Ping:** Hace un Ping al servidor y devuelve el tiempo que ha tardado en llegar la respuesta en milisegundos. Este método se bloquea hasta que llega la respuesta o hasta que se cumple el Timeout indicado. Si no hay respuesta o la petición falla devuelve -1.

Eventos:

- **Exception:** Este evento avisa de un error interno.
- **PeticiónByteConRespuesta:** Este evento avisa de que ha llegado un paquete de tipo matriz de bytes desde el servidor y además hay que devolver una respuesta del mismo tipo.

- **PeticionByteSinRespuesta:** Este evento avisa de que ha llegado un paquete de tipo matriz de bytes desde el servidor pero no se devuelve ninguna respuesta.
- **PeticionCadenaConRespuesta:** Este evento avisa de que ha llegado un paquete de tipo cadena de caracteres desde el servidor y además hay que devolver una respuesta del mismo tipo.
- **PeticionCadenaSinRespuesta:** Este evento avisa de que ha llegado un paquete de tipo cadena de caracteres desde el servidor pero no se devuelve ninguna respuesta.



CLIENTE UDP

Conecta con el servidor utilizando el protocolo UDP. Este protocolo no se recomienda para trabajar con el servidor ya que los paquetes de información que se pueden enviar son mucho más pequeños que en TCP. Tampoco se recomienda ya que puede haber problemas con los paquetes a través de internet. Este protocolo se recomienda en redes locales con paquetes de información pequeños.

Propiedades:

- **Conectado:** Propiedad de solo lectura de tipo Bool. Indica que la conexión con el servidor es correcta.
- **Tiempo:** Propiedad de solo lectura de tipo Int32. Devuelve la media de tiempo de peticiones en milisegundos de las últimas 10 peticiones.

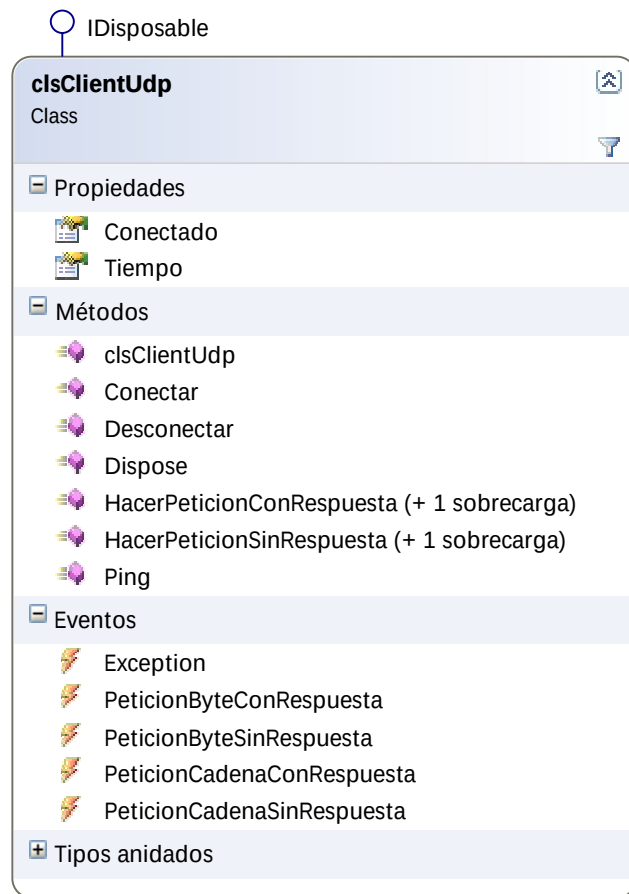
Métodos:

- **Conectar:** Se conecta a un servidor. Solo hay que indicar la dirección y puerto.
- **Desconectar:** Se desconecta del servidor.
- **Dispose:** Finaliza el cliente. Cierra la conexión abierta y termina los hilos internos de forma controlada.
- **HacerPeticiónConRespuesta:** Hace una petición al servidor y espera la respuesta. Puede hacer peticiones de 2 tipos: De cadena de texto o de matriz de bytes. Este método se bloquea hasta que llegue la respuesta o hasta que se cumpla el Timeout indicado. Se pueden hacer tantas peticiones en paralelo en distintos hilos como sea necesario.
- **HacerPeticiónSinRespuesta:** Hace una petición al servidor. Puede hacer dos tipos de peticiones: De cadena de texto o de matriz de bytes. Este método no se bloquea. Se pueden hacer tantas peticiones en paralelo en distintos hilos como sea necesario.
- **Ping:** Hace un Ping al servidor y devuelve el tiempo que ha tardado en llegar la respuesta en milisegundos. Este método se bloquea hasta que llega la respuesta o hasta que se cumple el Timeout indicado. Si no hay respuesta o la petición falla devuelve -1.

Eventos:

- **Exception:** Este evento avisa de un error interno.

- **PeticionByteConRespuesta:** Este evento avisa de que ha llegado un paquete de tipo matriz de bytes desde el servidor y además hay que devolver una respuesta del mismo tipo.
- **PeticionByteSinRespuesta:** Este evento avisa de que ha llegado un paquete de tipo matriz de bytes desde el servidor pero no se devuelve ninguna respuesta.
- **PeticionCadenaConRespuesta:** Este evento avisa de que ha llegado un paquete de tipo cadena de caracteres desde el servidor y además hay que devolver una respuesta del mismo tipo.
- **PeticionCadenaSinRespuesta:** Este evento avisa de que ha llegado un paquete de tipo cadena de caracteres desde el servidor pero no se devuelve ninguna respuesta.



REVISIONES

V1.0.0.0

Primera versión del programa estable.

FUNCIONAMIENTO INTERNO DEL PROTOCOLO

Protocolo TCP:

El funcionamiento interno es ligeramente diferente para protocolo TCP que para el UDP. El protocolo TCP al poder fraccionar la información en varios paquetes se le añade una cabecera con un inicio de paquete y su tamaño para que el receptor sepa cuantos datos debe esperar antes de empezar a interpretar los datos. Esto se hace en la tercera capa. Esta capa no es necesaria en UDP.

Primera capa

Si el paquete que se quiere enviar es una cadena de texto lo primero es convertir la cadena a una matriz de bytes utilizando la codificación UTF8. Si es una cadena de bytes no hay que hacer nada. Si es un Ping no hay datos. A la matriz de datos se le añade delante 5 bytes que son, el primer byte identifica el tipo de paquete (ver tabla 1) para que el receptor pueda interpretar los datos de forma correcta, los 4 bytes siguientes un numero de 32 bits con un ID que identifica el paquete de forma única. En el caso del Ping el paquete son 5 bytes ya que no hay datos.

Tipo	Número
Petición sin respuesta de tipo matriz de bytes	10
Petición sin respuesta de tipo cadena de texto	11
Petición con respuesta de tipo matriz de bytes	20
Respuesta a una petición hecha de tipo matriz de bytes	21
Petición con respuesta de tipo cadena de caracteres	22
Respuesta a una petición hecha de tipo cadena de caracteres	23
Petición de Ping	24
Respuesta de Ping	25

Tabla 1

En las peticiones el ID es un número que se genera de forma consecutiva y sin repetir ninguno. En las respuestas el ID es el mismo que traía la petición, de esta forma la clase que envió el paquete de petición puede saber cuál es la respuesta de dicho paquete. De esta manera se pueden hacer varias peticiones simultáneamente sin que se mezclen las respuestas.

Segunda capa

La segunda recibe una matriz de bytes de la primera capa y lo primero que hace es comprimirla. Lo segundo que hace es escoger entre los datos originales y los datos comprimidos el que menos espacio ocupa y lo envía a encriptar. Después le añade un byte delante que indica que tratamiento tiene el paquete (ver tabla siguiente). Por último lo que hace es enviar el paquete a la tercera capa.

Tipo	Número
Paquete sin encriptar, sin comprimir	4
Paquete sin encriptar, comprimido	3
Paquete encriptado, sin comprimir	2
Paquete encriptado y comprimido	1

Tabla 2

Tercera capa

La tercera capa recibe la matriz de bytes de la segunda capa y le añade 9 bytes delante. Estos 9 bytes definen el inicio de un paquete. Los 5 primeros bytes son fijos [67,82,69,84,65]. Los siguientes 4 bytes son un número de 32 bits que indican el tamaño total del paquete incluido los 9 bytes de la cabecera. Por último se envía el paquete al destinatario.

Protocolo UDP:

Para el protocolo UDP se establecen 2 capas. Estas son las mismas que las 2 primeras capas del TCP. La tercera capa que usa el TCP no es necesaria en UDP ya que en este protocolo no se puede fraccionar la información. Por lo tanto la capacidad de información de este protocolo es bastante inferior a la del TCP.

Primera capa

Si el paquete que se quiere enviar es una cadena de texto lo primero es convertir la cadena a una matriz de bytes utilizando la codificación UTF8. Si es una cadena de bytes no hay que hacer nada. Si es un Ping no hay datos. A la matriz de datos se le añade delante 5 bytes que son, el primer byte identifica el tipo de paquete (ver tabla 1) para que el receptor pueda interpretar los datos de forma correcta, los 4 bytes siguientes un número de 32 bits con un ID que identifica el paquete de forma única. En el caso del Ping el paquete son 5 bytes ya que no hay datos.

En las peticiones el ID es un número que se genera de forma consecutiva y sin repetir ninguno. En las respuestas el ID es el mismo que traía la petición, de esta forma la clase que envió el paquete de petición puede saber cuál es la respuesta de dicho paquete. De esta manera se pueden hacer varias peticiones simultáneamente sin que se mezclen las respuestas.

Segunda capa

La segunda recibe una matriz de bytes de la primera capa y lo primero que hace es comprimirla. Lo segundo que hace es escoger entre los datos originales y los datos comprimidos el que menos espacio ocupa y lo envía a encriptar. Después le añade un byte delante que indica que tratamiento tiene el paquete (ver tabla 2). Por último lo que hace es enviar el paquete al destinatario.